



UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS

FACULTAD DE INGENIERÍA

CURSO: 1AMA0726 – MATEMÁTICA COMPUTACIONAL

TEORÍA DE JUEGOS

INTEGRANTES	Código
Aguilar Flores, Marco Aurelio	u202272456
Castro Espinoza, Luis Alberto	u202245623
Gálvez Hinostroza, Ana Paula	u202285726
Mendoza Alva, Luciana	u202210723
Torres Luna, Miguel Ángel	u202271145

PROFESOR

Mattos Sueros, Edgard Raul

UPC, agosto de 2024

Contenido

Introducción	3
Objetivo	3
Metodología	3
Algoritmo de enumeración de soportes	3
Desarrollo del código	4
Librerías Utilizadas	4
Descripción General del Programa	4
Búsqueda de Respuestas Óptimas	5
Identificación del Equilibrio de Nash	5
Diagrama de flujo del programa ejecutable	6
Visualización de Resultados	7
Programa	7
Resultados.....	7
Conclusiones	8
Referencias Bibliográficas	9

Teoría de juegos

Introducción

A lo largo de la historia, las personas han tenido que tomar decisiones considerando no solo sus propios intereses, sino también las posibles acciones de quienes las rodean. Desde negociaciones comerciales y conflictos políticos hasta competencias deportivas y juegos de estrategia, el resultado final suele depender de las decisiones tomadas por varios participantes de manera simultánea. Esta necesidad de comprender y anticipar el comportamiento de los demás ha motivado el desarrollo de herramientas matemáticas capaces de analizar situaciones de interacción estratégica

Durante el siglo XX, el estudio de estos problemas adquirió gran relevancia gracias a los aportes de diversos matemáticos y economistas, destacando especialmente los trabajos de John Nash sobre los equilibrios en juegos no cooperativos. Sus investigaciones permitieron comprender mejor cómo los individuos pueden alcanzar situaciones estables en las que ninguna de las partes tiene incentivos para modificar su decisión de manera unilateral.

En este informe se presentan los principales fundamentos relacionados con la Teoría de Juegos en estrategias puras, el modelado matemático utilizado para su análisis y los procedimientos computacionales empleados para estudiar este tipo de problemas mediante herramientas de Matemática Computacional.

Objetivo

Desarrollar un programa elaborado en Python que resuelva juegos en estrategias puras con una interfaz intuitiva y amigable para facilitar su uso y comprensión.

Metodología

Algoritmo de enumeración de soportes

El programa utiliza el algoritmo de Enumeración de Soportes (Support Enumeration) para encontrar los equilibrios de Nash en un juego de dos jugadores. Primero, el usuario ingresa el número de estrategias y los pagos correspondientes para cada combinación de decisiones. A partir de esta información se construyen las matrices de pago de ambos jugadores. Luego, el algoritmo analiza todas las posibles combinaciones de estrategias que podrían formar parte de un equilibrio y calcula las

probabilidades con las que cada jugador debería elegir sus acciones. Posteriormente, verifica si dichas estrategias cumplen la condición fundamental de un equilibrio de Nash: que ningún jugador pueda mejorar su beneficio cambiando unilateralmente de estrategia mientras el otro mantiene su decisión. Una vez identificados los equilibrios, el programa los clasifica como estrategias puras o estrategias mixtas. Se considera un equilibrio en estrategias puras cuando cada jugador selecciona una única estrategia con probabilidad igual a 1; es decir, la decisión es completamente determinística. En cambio, se considera un equilibrio en estrategias mixtas cuando uno o ambos jugadores deben combinar varias estrategias asignándoles diferentes probabilidades para mantener el equilibrio. Finalmente, el sistema muestra las estrategias óptimas encontradas y los beneficios o pagos que obtiene cada jugador en el equilibrio identificado, proporcionando una interpretación clara de la solución del juego.

Desarrollo del código

Librerías Utilizadas

Para la implementación del programa se emplearon las siguientes bibliotecas:

NumPy permite trabajar eficientemente con matrices y operaciones algebraicas.

SciPy proporciona herramientas de optimización matemática que pueden emplearse para extender el análisis hacia estrategias mixtas.

Matplotlib permite representar gráficamente matrices de resultados y análisis realizados por el programa.

Figura 1

Importación de librerías principales del proyecto

```
import numpy as np
from scipy.optimize import linprog
import matplotlib.pyplot as plt
```

Nota. Código utilizado para importar las bibliotecas necesarias para el procesamiento numérico, optimización y visualización de resultados en Python. Elaboración propia.

Descripción General del Programa

El sistema fue desarrollado para recibir las ganancias de los jugadores mediante una interfaz interactiva. Posteriormente, la aplicación construye una matriz de pagos y

analiza las respuestas óptimas de cada participante con el objetivo de identificar los Equilibrios de Nash presentes en el juego.

Figura 2

Ingreso de la matriz de pagos

```
matriz_pagos = np.array([
    [(3,2), (1,1)],
    [(2,4), (0,3)]
])
```

Nota. Interfaz del programa utilizada para registrar las ganancias asociadas a cada combinación de estrategias de los jugadores. Elaboración propia.

Búsqueda de Respuestas Óptimas

El programa analiza cada columna y cada fila para identificar las mejores respuestas posibles. De manera similar, se calculan las mejores respuestas del jugador B utilizando los pagos correspondientes.

Figura 3

Determinación de las respuestas óptimas

```
for j in range(columnas):
    mejor_respuesta = max(
        matriz[i][j][0]
        for i in range(filas)
    )
```

Nota. Resultado generado por el sistema al identificar las estrategias que maximizan las ganancias de cada jugador para una matriz de pagos determinada. Elaboración propia.

Identificación del Equilibrio de Nash

Una vez determinadas las respuestas óptimas de ambos jugadores, el programa busca coincidencias entre ellas. Las posiciones encontradas representan Equilibrios de Nash en estrategias puras.

Figura 4

Identificación del Equilibrio de Nash

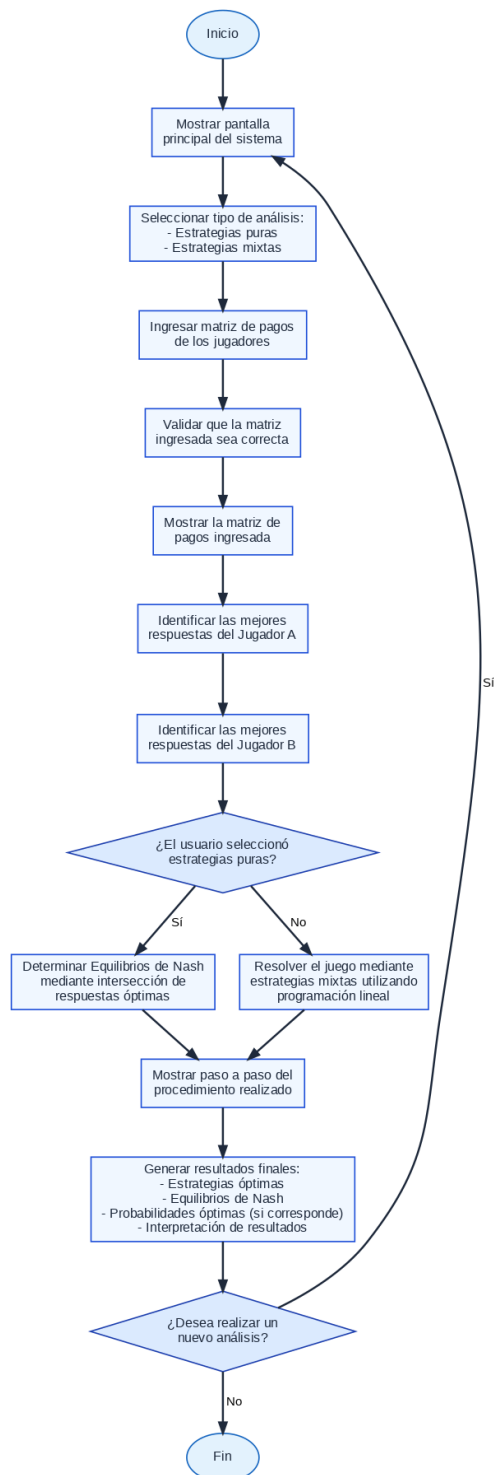
```
if respuesta_A and respuesta_B:
    equilibrios.append((i, j))
```

Nota. El sistema identifica la combinación de estrategias que constituye un Equilibrio de Nash, donde ningún jugador tiene incentivos para cambiar unilateralmente su decisión. Elaboración propia.

Diagrama de flujo del programa ejecutable

Figura 5

Diagrama de flujo del sistema para la identificación de Equilibrios de Nash



Nota. El diagrama muestra la secuencia de operaciones ejecutadas por el programa, desde el ingreso de la matriz de pagos hasta la determinación y visualización de los Equilibrios de Nash encontrados. Elaboración propia.

Visualización de Resultados

Los resultados obtenidos se muestran al usuario mediante tablas y mensajes descriptivos. La visualización facilita la interpretación de los resultados obtenidos por el algoritmo.

Figura 5

Selección del método de estrategias para el análisis de la matriz de ganancias

```
plt.imshow(matriz_resultados)
plt.title("Equilibrios de Nash")
plt.show()
```

Nota. Pantalla de presentación del sistema utilizado para el análisis de juegos en estrategias puras y búsqueda de Equilibrios de Nash. Elaboración propia.

Programa

En esta etapa se presenta el desarrollo completo del proyecto, implementado en Python para el análisis y resolución de juegos en estrategias puras y mixtas. Como parte de la solución propuesta, se diseñó una interfaz web que permite al usuario interactuar con el sistema de manera sencilla, ingresar los datos del juego y obtener los Equilibrios de Nash correspondientes. La plataforma desarrollada está disponible en el siguiente enlace: <https://criptografrsa.vercel.app/>

Resultados

Luego de implementar el algoritmo para el análisis de juegos en estrategias puras, se realizaron diversas pruebas utilizando matrices de pagos con el fin de verificar el correcto funcionamiento del programa. Los resultados obtenidos permitieron identificar de manera automática las respuestas óptimas de cada jugador y los posibles Equilibrios de Nash presentes en cada escenario analizado. Los valores obtenidos fueron consistentes con los resultados esperados a partir del análisis teórico realizado previamente.

Durante las pruebas se observó que el programa facilita significativamente el estudio de los juegos estratégicos, ya que automatiza un proceso que puede resultar laborioso cuando se realiza manualmente. La identificación de las mejores respuestas y de los puntos de equilibrio se efectuó de forma rápida y precisa, permitiendo analizar

diferentes configuraciones de estrategias sin necesidad de repetir cálculos de manera manual.

Asimismo, los resultados obtenidos evidencian la estrecha relación entre la Teoría de Juegos y la Matemática Computacional. El uso de herramientas de programación permitió representar matrices de pagos, procesar información estratégica y mostrar resultados de manera organizada. Esta integración entre teoría y programación contribuye a una mejor comprensión de conceptos como respuesta óptima y Equilibrio de Nash, facilitando la interpretación de problemas de toma de decisiones en contextos reales

Conclusiones

El desarrollo del proyecto permitió comprender los fundamentos de la Teoría de Juegos y su aplicación en el análisis de situaciones donde las decisiones de un participante influyen en los resultados de los demás. A través del estudio de las matrices de pagos y del Equilibrio de Nash, fue posible analizar cómo los jugadores pueden llegar a situaciones estables en las que no existe incentivo para modificar unilateralmente sus estrategias

Asimismo, la implementación computacional facilitó la identificación automática de las respuestas óptimas y de los Equilibrios de Nash en estrategias puras. Esto permitió reducir el tiempo requerido para realizar los cálculos y verificar que los resultados obtenidos por el programa coinciden con los fundamentos matemáticos estudiados durante el desarrollo del proyecto.

Finalmente, el proyecto evidenció la importancia de las herramientas computacionales como apoyo para el análisis de problemas matemáticos. La utilización de Python y de bibliotecas especializadas permitió desarrollar una solución capaz de representar, analizar y visualizar información estratégica de manera eficiente, contribuyendo al aprendizaje y aplicación de conceptos propios de la Matemática Computacional

Referencias Bibliográficas

- Axelrod, R. & Hamilton, W. D. (1981). The Evolution of Cooperation. *Science*, 211(4489), 1390-1396.
- Basar, T. & Olsder, G. J. (1999). Dynamic Noncooperative Game Theory. *Society for Industrial and Applied Mathematics*.
- Berg, J., Dickhaut, J. & McCabe, K. (1995). Trust, Reciprocity, and Social History. *Games and Economic Behavior*, 10(1), 122-142.
- Cerdá, E., Jimeno, J. L., & Pérez, J. (2004). *Teoría de juegos* (Vol. 53). Madrid, Spain: Pearson Educación.
- Fudenberg, D. & Tirole, J. (1991). *Game Theory*. The MIT Press.
- Güth, W., Schmittberger, R. & Schwarze, B. (1982). An Experimental Analysis of Ultimatum Bargaining. *Journal of Economic Behavior and Organization*, 3(4), 367-388.
- Krishna, V. (2002). *Auction Theory*. Academic Press.
- Monsalve, S. (2003). John Nash y la teoría de juegos. *Lecturas matemáticas*, 24(2), 137-149.
- Riker, W. H. (1962). The Theory of Political Coalitions. *Yale University Press*.